

Ottodot Take Home Task

peacewalker122

1 Ottodot Take Home Task

What I built



trial-class booking app. Parents book trial seats for their children and pay online. Stack is Bun, TypeScript, Postgres, and HTMX — no frontend framework, pages render on the server.

Requirement



1. A parent to choose a child and pick an available trial class.
2. A parent to submit a trial booking.
3. A mock payment step or payment result to be recorded.
4. The booking status to be shown after submission.
5. An admin or teacher to see the trial class roster or a simple roster API/output.

Problem



Booking has four classic problems.

1. the same child gets booked twice.
2. more students confirmed than seats — overbooking.
3. two payments arrive at the same moment for the last seat.
4. payment fails halfway and the booking gets stuck.

Based on the requirement and the problem, we can infer that the app has 3 business invariant:

1. For every class, $\text{booked}(s_1) \implies s_1 = s_2$
2. $\text{Confirmed Booking} \leq \text{Class maximum capacity}$
3. For all $\text{booking} \subset \text{confirmedRoster}(\text{class}) \implies \text{exist some, successful immutable paymentEvent for booking.}$

How it prevented



1. Duplicate Bookings

the database has a unique rule on (child, class) column. Same child and class can never have two rows. If two requests arrive together, the loser reuses the winner's booking instead of crashing.

2. Overbooking

each class has a `maximum_capacity`. The seat is taken with one conditional update — count goes up only if it is still below the cap. Two workers cannot both win. The database also rejects any count above the cap, so even a bug in my code cannot overbook.

3. Last Seat Race

The user A from the problem will win, because when they already booked it and already at the state of “held” lasted for 10 minutes. With this, if user b try to book at t2, user b will get

How it prevented (ii)



notification that the class is already full. If the booking payment is successful the kid will be appended to the roster, if payment fail / expired, it will release the held seat.

4. Failed Payment

the booking waits for payment first and only joins the roster after payment succeeds. If payment fails, the parent can re-book — the old failed attempt stays as history and a new invoice is created. Retried callbacks and retried worker jobs are idempotent, so nothing is ever counted twice.

How I build it



First the database: tables for parents, students, classes, bookings, payment attempts, an append-only payment event log, and an outbox queue. Rules live in the database, not in application code.

Then the booking service, then the payment callback intake with signature check, then the background worker that confirms or refunds. UI last: booking form, live status cards, roster, payment ledger.

How I proved it works



Six integration tests, all passing: dedupe, callback replay, and the last-seat race. A script checks the invariants — no duplicates, no over-capacity, no unpaid booking on the roster. I also tested the real HTTP flow: book, fail the payment, press try again, succeed, and land confirmed.